I'll explain these **important JavaScript exam topics in detailed 8–10 mark answers** so you can directly write them in exams or viva.

---

1. DOM vs HTML (Difference)

Feature	HTML	DOM
Meaning	HyperText Markup Language used to structure web pages	Document Object Model – programming interface for HTML
Nature	Static	Dynamic
Created by	Written by developer	Created by browser
Representation	Text-based markup	Tree structure of objects
Manipulation	Cannot change after loading	Can be modified using JavaScript

Example HTML

```
<html>
<body>
<h1>Hello</h1>
</body>
</html>
```

DOM Representation

```
Document
  └── html
       └── body
            └── h1
                 └── "Hello"
```

Uses of DOM

JavaScript can:

- Change content
- Change style
- Add or delete elements
- Handle events
- Create dynamic web pages

Example

```
document.getElementById("title").innerText = "New Text";
```

---

2. innerText vs innerHTML vs textContent

These are **DOM properties used to read or change content of HTML elements.**

Property	Description	Includes HTML Tags	Hidden Text
innerText	Returns visible text only	❌ No	❌ No
textContent	Returns all text	❌ No	✓ Yes
innerHTML	Returns HTML markup	✓ Yes	✓ Yes

Example

HTML

```
<div id="box">
  Hello <b>World</b>
</div>
```

innerText

```
document.getElementById("box").innerText
```

Output

```
Hello World
```

---

textContent

```
document.getElementById("box").textContent
```

Output

```
Hello World
```

(includes hidden text too)

---

innerHTML

```
document.getElementById("box").innerHTML
```

Output

```
Hello <b>World</b>
```

---

3. DOM Modification (Creating, Adding, Removing Elements)

DOM allows us to **modify the HTML structure dynamically.**

Steps

1 Create element
2 Add content
3 Append element
4 Remove element

Create Element

```
let div = document.createElement("div");
```

Add Content

```
div.innerText = "Hello";
```

Append Element

```
document.body.appendChild(div);
```

Remove Element

```
div.remove();
```

---

4. Example (10 Marks) – To-Do List using DOM

HTML

```
<input type="text" id="task">
<button onclick="addTask()">Add</button>

<ul id="list"></ul>
```

JavaScript

```
function addTask(){

let task = document.getElementById("task").value;

let li = document.createElement("li");

li.innerText = task;

document.getElementById("list").appendChild(li);

}
```

Explanation

1 User enters task
2 Button click triggers function
3 New `<li>` element created
4 Task text inserted
5 Element added to list

---

5. addEventListener and removeEventListener

addEventListener()

Used to attach event to an element.

Syntax

```
element.addEventListener(event, function);
```

Example

```
let btn = document.getElementById("btn");

btn.addEventListener("click", function(){
    alert("Button Clicked");
});
```

removeEventListener()

Removes previously attached event.

Example

```
function show(){
    alert("Hello");
}

btn.addEventListener("click", show);

btn.removeEventListener("click", show);
```

---

6. Event Bubbling and Event Capturing

When an event occurs, it travels through DOM elements.

Event Bubbling

Event moves **from child → parent**.

Example

```
<div id="parent">
   <button id="child">Click</button>
</div>
document.getElementById("child").addEventListener("click",()=>{
 console.log("Child clicked");
});

document.getElementById("parent").addEventListener("click",()=>{
 console.log("Parent clicked");
});
```

Output

```
Child clicked
Parent clicked
```

Event Capturing

Event moves **parent → child**.

```
element.addEventListener("click", handler, true);
```

`true` enables capturing.

---

7. Call Stack and Execution Context

JavaScript is **single-threaded**, meaning it executes one task at a time.

Execution Context

Environment where JavaScript code executes.

Types

1 Global Execution Context
2 Function Execution Context

Example

```
function a(){
  b();
}

function b(){
  console.log("Hello");
}

a();
```

Call Stack

Stack structure where function calls are stored.

Execution

```
Call Stack
```

```
Global
   ↓
a()
   ↓
b()
```

Then functions return in **LIFO order (Last In First Out)**.

---

8. Timers (Output Based)

Timers run asynchronously.

setTimeout

Runs once after delay.

```
setTimeout(()=>{
 console.log("Hello");
},2000);
```

---

setInterval

Runs repeatedly.

```
setInterval(()=>{
 console.log("Running");
},1000);
```

---

9. Callback, Promise and Promise Lifecycle

Callback

Function passed as argument to another function.

Example

```
setTimeout(function(){
 console.log("Task done");
},2000);
```

Callback Hell

Nested callbacks.

Example

```
login(()=>{
 getUser(()=>{
  getOrders(()=>{
   processOrder()
  })
 })
})
```

Problems

- Hard to read
- Difficult debugging

Promise

Promise represents **future result of async operation**.

Syntax

```
let promise = new Promise((resolve,reject)=>{
   resolve("Success");
});
```

Promise Lifecycle

1 Pending

Initial state.

2 Fulfilled

Operation successful.

3 Rejected

Operation failed.

Promise Consumers

then()

Runs when resolved.

catch()

Handles error.

finally()

Runs always.

Example

```
promise
.then(res => console.log(res))
.catch(err => console.log(err))
.finally(()=>console.log("Completed"));
```

---

10. Promise Chaining

Multiple `.then()` methods used sequentially.

Example

```
fetchData()
.then(data => process(data))
.then(result => display(result))
.catch(err => console.log(err));
```

Advantages

- Cleaner than callbacks
- Avoids callback hell

---

11. Event Loop (Important 10 Mark Question)

JavaScript handles asynchronous tasks using **Event Loop**.

Main Components

1 Call Stack
2 Web APIs
3 Callback Queue
4 Microtask Queue
5 Event Loop

Working

1 JS executes synchronous code in call stack
2 Async tasks move to Web APIs
3 After completion they move to queues
4 Event loop checks call stack
5 Executes queued tasks

Priority

```
Microtask Queue (Promises)
↓
Macrotask Queue (setTimeout)
```

---

12. Local Storage vs Session Storage vs Cookies

Feature	Local Storage	Session Storage	Cookies
Storage Size	~5MB	~5MB	~4KB
Expiry	No expiry	Deleted after tab close	Expiry set by developer
Access	JS only	JS only	JS + Server
Use Case	Save preferences	Temporary data	Authentication

Local Storage Example

```
localStorage.setItem("name","John");

localStorage.getItem("name");
```

Session Storage Example

```
sessionStorage.setItem("user","Bhavya");
```

Cookies Example

```
document.cookie = "username=Bhavya";
```

Quick Revision (Very Important Topics for Exam)

1. DOM vs HTML
2. innerText vs innerHTML vs textContent
3. DOM Manipulation (Create, Append, Remove)
4. Event Handling (addEventListener)
5. Event Bubbling vs Capturing
6. Call Stack and Execution Context
7. Timers (setTimeout / setInterval)
8. Callback → Promise → Promise Chaining
9. Event Loop
10. LocalStorage vs SessionStorage vs Cookies

If you want, I can also give you **20 JavaScript output questions for Event Loop, Promises and Timers** (these are very commonly asked in exams and interviews).

Q1. Difference between `var`, `let`, and `const`

Feature	var	let	const
Scope	Function scoped	Block scoped	Block scoped
Re-declaration	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed
Hoisting	Yes (initialized with undefined)	Yes but in TDZ	Yes but in TDZ

Example

```
var a = 10;
var a = 20; // allowed

let b = 10;
b = 20; // allowed
// let b = 30; ❌ not allowed

const c = 10;
// c = 20 ❌ error
```

Key Points

- `var` is old ES5 variable declaration.
- `let` and `const` introduced in **ES6**.
- `const` must be initialized during declaration.

Q2. Types of Operators in JavaScript

1. Arithmetic Operators

Used for mathematical calculations.

Operator	Example
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponent

```
let a = 10;
let b = 5;
console.log(a + b); // 15
```

---

2. Assignment Operators

Assign values to variables.

Operator	Example
=	x = 10
+=	x += 5

Operator	Example
-=	x -= 5
*=	x *= 5

```
let x = 10;
x += 5; // x = 15
```

---

3. Comparison Operators

Compare two values and return **true or false**.

Operator	Example
==	equal
===	strict equal
!=	not equal
>	greater than

---

4. Logical Operators

Used for logical conditions.

Operator	Meaning
&&	AND
!	NOT

```
if(age > 18 && age < 60)
```

---

5. Bitwise Operators

Operate on binary numbers.

Operator	Meaning
&	AND
|	OR
^	XOR
~	NOT

---

6. Ternary Operator

Short form of `if-else`.

```
condition ? trueValue : falseValue
```

Example

```
let age = 18;
let result = age >= 18 ? "Adult" : "Minor";
```

---

7. Type Operators

Used to check variable type.

Operator	Example
typeof	typeof x
instanceof	obj instanceof Array

Q3. Difference between == and ===

Operator	Meaning
==	Loose equality
===	Strict equality

== (Loose Equality)

Compares **value only** and converts types.

```
5 == "5" // true
```

=== (Strict Equality)

Compares **value AND datatype**.

```
5 === "5" // false
```

Always prefer ===.

Q4. Hoisting and TDZ

Hoisting

JavaScript moves **variable and function declarations to the top of the scope**.

Example:

```
console.log(a);
var a = 5;
```

Internally becomes:

```
var a;
console.log(a); // undefined
a = 5;
```

TDZ (Temporal Dead Zone)

The time between **variable declaration and initialization** where the variable **cannot be accessed**.

Example

```
console.log(a); // error
let a = 10;
```

TDZ exists for **let and const**.

Q5. Important Array Methods

map()

Transforms each element and returns new array.

```
let arr = [1,2,3];
let result = arr.map(x => x * 2);
```

Output

```
[2,4,6]
```

---

filter()

Filters elements based on condition.

```
let arr = [1,2,3,4];
let result = arr.filter(x => x > 2);
```

Output

```
[3,4]
```

---

reduce()

Reduces array to a single value.

```
let arr = [1,2,3,4];
let sum = arr.reduce((acc, curr) => acc + curr, 0);
```

Output

```
10
```

---

map vs forEach

Feature	map	forEach
Return value	New array	Undefined
Used for	Transformation	Iteration

---

find vs filter

Method	Output
find()	First matching element
filter()	All matching elements

---

some vs every

Method	Meaning
some()	At least one condition true
every()	All conditions true

Q6. String Methods and Object Methods

String Methods

Method	Description
length	String length
toUpperCase()	Uppercase
toLowerCase()	Lowercase
includes()	Check substring
slice()	Extract part

Example

```
let str = "JavaScript";
console.log(str.slice(0,4));
```

Output

```
Java
```

Object Methods

Method	Description
Object.keys()	Returns keys
Object.values()	Returns values
Object.entries()	Key-value pairs
Object.assign()	Copy object

Example

```
let obj = {name:"John", age:25};

console.log(Object.keys(obj));
```

Output

```
["name","age"]
```

Q7. Types of Functions

Regular Function

```
function add(a,b){
 return a+b;
}
```

Arrow Function

```
const add = (a,b) => a+b;
```

First Class Function

Functions treated as **values**.

```
function greet(){
 console.log("Hello");
}

let say = greet;
say();
```

Higher Order Function

Function that **accepts another function**.

Example

```
map()
filter()
reduce()
```

Callback Function

Function passed as argument.

```
setTimeout(function(){
 console.log("Hello");
},1000);
```

IIFE

Runs immediately.

```
(function(){
 console.log("IIFE");
})();
```

Anonymous Function

Function without name.

```
setTimeout(function(){
 console.log("Hello");
});
```

Q8. Difference between `setTimeout()` and `setInterval()`

Method	Meaning
setTimeout	Runs once
setInterval	Runs repeatedly

Example

```
setTimeout(()=>console.log("Hello"),2000);
```

Q9. DOM vs HTML

HTML	DOM
Static structure	Dynamic object
Written in file	Created by browser
Markup	Programming interface

DOM allows:

- change content
- change style
- add/remove elements
- handle events

Q10. textContent vs innerHTML vs innerText

Property	Meaning
textContent	Returns text including hidden
innerText	Only visible text
innerHTML	Includes HTML tags

Example

```
element.innerHTML = "<b>Hello</b>";
```

Q11. DOM Selectors

getElementById

Returns single element.

```
document.getElementById("id")
```

---

getElementsByClassName

Returns **HTML collection**.

```
document.getElementsByClassName("class")
```

---

querySelector

Returns first matching element.

```
document.querySelector(".class")
```

---

querySelectorAll

Returns **NodeList**.

```
document.querySelectorAll(".class")
```

Q12. Create, Append, Remove Element

Create

```
let div = document.createElement("div");
```

Append

```
document.body.appendChild(div);
```

Remove

```
div.remove();
```

Q13. Event Handling

Events are **user interactions**.

Examples

- click
- hover
- submit

Example

```
btn.addEventListener("click", function(){
 console.log("Clicked");
});
```

Q14. classList Methods

Method	Meaning
add()	Add class
remove()	Remove class
toggle()	Add/remove automatically

Example

```
element.classList.toggle("active");
```

Q15. Form Validation using DOM

Events used:

Event	Meaning
onclick	click
onchange	input change
onmouseover	mouse hover
onsubmit	form submit

Example

```
form.onsubmit = function(){
 if(name.value === ""){
  alert("Name required");
 }
}
```

Q16. Event Bubbling and Capturing

Event Bubbling

Event travels **from child → parent**.

Example

```
button → div → body
```

Event Capturing

Event travels **parent → child**.

---

preventDefault()

Stops default behavior.

Example

```
event.preventDefault();
```

Used in forms to prevent page reload.

Q17. Execution Context and Call Stack

Execution Context

Environment where JS code runs.

Types

1 Global
2 Function

Call Stack

Stack structure where functions execute.

Example

```
main()
  ↓
function1()
  ↓
function2()
```

Last In → First Out.

Q18. Callback Hell

When callbacks are nested deeply.

Example

```
login(()=>{
 getUser(()=>{
   getOrders(()=>{
     getDetails()
   })
 })
})
```

Problems

- Hard to read
- Hard to debug
- Pyramid structure

Solution

- Promises
- Async Await

Q19. Promise Lifecycle

Promise Definition

Promise is an object representing **future completion or failure of an asynchronous operation**.

States

1 Pending
2 Fulfilled
3 Rejected

Consumers

```
.then()
.catch()
.finally()
```

Example

```
let promise = new Promise((resolve,reject)=>{
 let success = true;

 if(success)
  resolve("Success");
 else
  reject("Error");
});
```

```
promise
.then(res=>console.log(res))
.catch(err=>console.log(err))
.finally(()=>console.log("Done"));
```

Q20. Event Loop

JavaScript is **single-threaded**.

Components

1 Call Stack
2 Web APIs
3 Callback Queue
4 Event Loop

Event loop checks:

```
If call stack empty
↓
Move tasks from queue
↓
Execute
```

Q21. Promise Chaining

Multiple `.then()` calls.

Example

```
fetchData()
.then(data => process(data))
.then(result => display(result))
```

Disadvantages

- Hard to debug
- Nested complexity

Q22. Async / Await

Cleaner way to write async code.

Example

```
async function getData(){
 let res = await fetch(url);
 let data = await res.json();
 console.log(data);
}
```

Advantages

- readable
- synchronous style
- easier debugging

Q23 Fetch API

Fetch sends **HTTP request**.

Example

```
fetch("https://api.com/data")
.then(res => res.json())
.then(data => console.log(data))
```

Steps

1 Send request
2 Receive response
3 Convert JSON
4 Use data

Output Based Questions (Event Loop)

Q23 Output

```
console.log("A");
```

```
setTimeout(() => {
  console.log("B");
}, 1000);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Output

```
A
D
C
B
```

Reason

1 A executes
2 setTimeout goes to WebAPI
3 Promise goes to microtask queue
4 D executes
5 Microtask executes → C
6 Macrotask executes → B

Q24 Output

```
A
E
C
D
B
```

Reason

Order

1 A
2 E
3 Promise (microtask)
4 setTimeout (macrotask)

Q25 Output

```
1
4
2
3
```

Reason

Promises execute after synchronous code.

Q26 Output

```
Promise 1
Timeout 1
Promise inside Timeout
```

Reason

1 Promise microtask runs first
2 Timeout executes
3 Promise inside timeout runs

Q27 Output

```
1
5
3
4
2
```

Reason

1 sync → 1,5
2 promise microtasks → 3,4
3 macrotask → 2

Q28 Output

```
A
B
C
```

E
D

Reason

1 A
2 Promise executor runs → B
3 resolve called
4 C executes
5 E
6 then executes → D

Weather

```
<!-- https://home.openweathermap.org/api_keys -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Lab 2 - Async Weather Tracker</title>
    <style>
        /* KEEPING YOUR EXACT UI STYLE */
        body {
            background: linear-gradient(#1e3a8a, #0f172a);
            color: white;
            font-family: sans-serif;
            display: flex;
            justify-content: center;
            padding: 20px;
            min-height: 100vh;
        }
        .container { width: 100%; max-width: 800px; }

        .card {
            background: white;
            color: #333;
            padding: 20px;
            border-radius: 10px;
```

```
            margin-bottom: 20px;
            box-shadow: 0 4px 10px rgba(0,0,0,0.3);
        }

        .grid { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; }

        input { padding: 10px; width: 70%; border-radius: 5px; border: 1px solid #ccc;
}
        button {
            padding: 10px;
            background: #e67e22;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        .hist-btn {
            background: #e67e22;
            color: white;
            border: none;
            margin: 5px 5px 0 0;
            padding: 5px 12px;
            border-radius: 20px;
            font-size: 12px;
            cursor: pointer;
        }

        .row { display: flex; justify-content: space-between; padding: 8px 0; border-
bottom: 1px solid #eee; }
        .row b { color: #555; }

        .console {
            background: #0f172a;
            color: #94a3b8;
            padding: 15px;
            border-radius: 5px;
            font-family: monospace;
            height: 120px;
            overflow-y: auto;
        }
    </style>
</head>
<body>

    <div class="container">
        <!-- Added <u> tag for underline -->
        <h1 style="text-align: center;">☀ <u>Async Weather Tracker</u></h1>
```

```
        <div class="grid">
            <!-- LEFT COLUMN -->
            <div class="card">
                <h3>Search City</h3>
                <input type="text" id="cityInput" placeholder="Enter city name...">
                <button id="searchBtn">Search</button>

                <hr> <!-- Added horizontal line -->
                <h4>Search History</h4>
                <div id="historyList"></div>
            </div>

            <!-- RIGHT COLUMN -->
            <div class="card">
                <h3>Weather Info</h3>
                <div id="weatherInfo">
                    <!-- Added <i> for italics -->
                    <p style="text-align: center; color: #999;"><i>Search a city to
see details</i></p>
                </div>
            </div>
        </div>

        <!-- BOTTOM CONSOLE -->
        <div class="card">
            <h3>Console - Event Loop Analysis</h3>
            <hr>
            <div id="logBox" class="console"></div>
        </div>
    </div>

    <script>
        // Simple function to display logs in our black box
        function addLog(text) {
            var logBox = document.getElementById('logBox');
            logBox.innerHTML = logBox.innerHTML + "<div> > " + text + "</div>";
        }

        // The Main Async function for fetching weather
        async function getWeatherData(city) {
            var infoDiv = document.getElementById('weatherInfo');
            var logBox = document.getElementById('logBox');

            if (city == "") {
                alert("Please enter a city!");
                return;
            }
```

```
            logBox.innerHTML = ""; // Clear console for new search

            // --- DEMONSTRATING EVENT LOOP ---
            addLog("1. Sync: Code Started");

            // Macrotask (runs last)
            setTimeout(function() {
                addLog("4. Macrotask: setTimeout finished");
            }, 0);

            // Microtask (runs after sync code)
            Promise.resolve().then(function() {
                addLog("3. Microtask: Promise finished");
            });

            addLog("2. Sync: Fetching from API...");

            try {
                var apiKey = "919d271dac1bb5318b47f53499249864";
                var url = "https://api.openweathermap.org/data/2.5/weather?q=" + city
+ "&units=metric&appid=" + apiKey;

                // Using await for fetch
                var response = await fetch(url);

                if (!response.ok) {
                    throw new Error("City not found");
                }

                var data = await response.json();

                // Updating UI with results
                infoDiv.innerHTML = `
                    <div class="row"><b>City</b> <span>${data.name}</span></div>
                    <div class="row"><b>Temp</b>
<span>${data.main.temp}°C</span></div>
                    <div class="row"><b>Humidity</b>
<span>${data.main.humidity}%</span></div>
                    <div class="row"><b>Wind Speed</b> <span>${data.wind.speed}
m/s</span></div>
                `;

                saveHistory(data.name);
                addLog("5. Async: Weather data loaded");

            } catch (err) {
```

```
                infoDiv.innerHTML = "<p style='color:red; text-align:center;'>Error: "
+ err.message + "</p>";
                addLog("Error occurred: " + err.message);
            }
        }

        // Function to save cities to Local Storage
        function saveHistory(cityName) {
            var history = JSON.parse(localStorage.getItem('myHistory')) || [];

            // If city is not in list, add it
            if (history.indexOf(cityName) === -1) {
                history.push(cityName);
                localStorage.setItem('myHistory', JSON.stringify(history));
                showHistory();
            }
        }

        // Function to show history buttons
        function showHistory() {
            var history = JSON.parse(localStorage.getItem('myHistory')) || ["Delhi",
"Mumbai", "London"];
            var historyList = document.getElementById('historyList');
            historyList.innerHTML = "";

            // Standard for loop to create buttons
            for (var i = 0; i < history.length; i++) {
                var btn = document.createElement('button');
                btn.className = "hist-btn";
                btn.innerText = history[i];

                // Set click for each button
                let currentCity = history[i];
                btn.onclick = function() {
                    getWeatherData(currentCity);
                };
                historyList.appendChild(btn);
            }
        }

        // Event listener for the Search button
        document.getElementById('searchBtn').onclick = function() {
            var inputVal = document.getElementById('cityInput').value;
            getWeatherData(inputVal);
        };

        // Load history when page opens
        window.onload = showHistory;
```

```
    </script>
</body>
</html>
```

1

```
<!DOCTYPE html>
<html>
<head>
    <title>Easy Event Dashboard</title>
    <style>
        /* CSS - This makes the page look pretty */
        body {
            background-color: #f0f2f5; /* Light grey background */
            font-family: Arial, sans-serif;
            display: flex;
            flex-direction: column;
            align-items: center; /* Puts everything in the middle */
            padding: 20px;
        }

        .box {
            background: white;
```

```
            padding: 20px;
            border-radius: 10px;
            width: 400px;
            box-shadow: 0px 4px 10px rgba(0,0,0,0.1); /* Adds a soft shadow */
            margin-bottom: 20px;
        }

        h2 { color: #333; text-align: center; }

        /* Style for the inputs where you type */
        input, select {
            width: 100%;
            padding: 10px;
            margin: 10px 0;
            border: 1px solid #ccc;
            border-radius: 5px;
            box-sizing: border-box; /* Makes sure padding doesn't break width */
        }

        /* The Add Button */
        .add-btn {
            background-color: #28a745; /* Green */
            color: white;
            border: none;
            width: 100%;
            padding: 10px;
            border-radius: 5px;
            cursor: pointer;
            font-weight: bold;
        }

        /* The Delete Button */
        .del-btn {
            background-color: #dc3545; /* Red */
            color: white;
            border: none;
            padding: 5px 10px;
            border-radius: 4px;
            cursor: pointer;
            float: right; /* Moves button to the right side of the card */
        }

        /* This is how a single event card looks */
        .event-card {
            background: #fff;
            border-left: 5px solid #007bff; /* Blue strip on the side */
            padding: 15px;
            margin-top: 10px;
```

```
            border-radius: 5px;
            box-shadow: 0px 2px 5px rgba(0,0,0,0.05);
        }
    </style>
</head>
<body>

    <h1>My Event Dashboard</h1>

    <!-- SECTION 1: The Form to enter data -->
    <div class="box">
        <h2>Add New Event</h2>
        <input type="text" id="eventName" placeholder="Enter Event Name (e.g. Physics
Lab)">
        <select id="eventCategory">
            <option>School</option>
            <option>Work</option>
            <option>Personal</option>
        </select>
        <button class="add-btn" onclick="addEvent()">Save Event</button>
    </div>

    <!-- SECTION 2: The List where events appear -->
    <div class="box">
        <h2>My Saved Events</h2>
        <div id="displayArea">
            <p style="color: gray; text-align: center;">No events saved yet.</p>
        </div>
    </div>

    <!-- SECTION 3: Keyboard Tracker (Just for fun) -->
    <div class="box" style="text-align: center;">
        <p>Last Key You Pressed: <b id="keyText">None</b></p>
    </div>

    <script>
        // JAVASCRIPT - This adds the "logic" (makes the buttons work)

        // 1. Function to add an event
        function addEvent() {
            // Get the text from the Input box
            var name = document.getElementById("eventName").value;
            // Get the choice from the Select box
            var category = document.getElementById("eventCategory").value;
            // Find the area where we want to put the new card
            var display = document.getElementById("displayArea");

            // Check if the user typed anything
```

```
            if (name == "") {
                alert("Please type an event name first!");
                return; // Stop here if empty
            }

            // If this is the first event, clear the "No events saved" text
            if (display.innerHTML.includes("No events saved")) {
                display.innerHTML = "";
            }

            // Create a new "div" (a box) for the event
            var newCard = document.createElement("div");
            newCard.className = "event-card"; // Give it the pretty CSS style

            // Fill the box with text and a Delete button
            newCard.innerHTML = "<b>" + name + "</b> (" + category + ")" +
                                '<button class="del-btn"
onclick="deleteMe(this)">Delete</button>';

            // Put this new box into our display area
            display.appendChild(newCard);

            // Clear the input box so you can type the next one
            document.getElementById("eventName").value = "";
        }

        // 2. Function to delete an event
        function deleteMe(button) {
            // "button.parentElement" is the event-card
            button.parentElement.remove();
        }

        // 3. Simple Keyboard Tracker
        window.onkeydown = function(event) {
            // Find the text spot and change it to the key you pressed
            document.getElementById("keyText").innerText = event.key;
        };
    </script>

</body>
</html>
```

Forms-

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>forms in js</title>
</head>
<body>
<form action="">
    <input type="text" name="username" placeholder="Enter Username"><br><br>
    <input type="password" name="password" placeholder="Enter Password"><br><br>
    <input type="submit" value="Login">
</form>
</body>
<script>
    const validUser = 'admin';
const validPass = 'secret';

document.querySelector('form').addEventListener('submit', e => {
  e.preventDefault();
  const { username, password } = e.target;
  if (username.value === validUser && password.value === validPass) {
    alert('Welcome!');
    // maybe window.location.href = 'dashboard.html';
  } else {
    alert('Invalid credentials');
  }
});
document.querySelector("form").addEventListener("submit", function(event){
    event.preventDefault();
    let username = event.target.username.value;
    let password = event.target.password.value;
    alert("Username: " + username + "\nPassword: " + password);
});
</script>
</html>
<!-- 1. wrap all input tags in a form tag -->
<!-- 2.use input type="submit" for making button -->
<!-- 3. always select form and apply event -->
 <!-- 4. always apply submit event -->
 <!-- 5. use event.preventDefault() -->
```